

I got a project with the below description:

AI agent (FikseAgent)

Purpose: Streamline human tasks like order creation by a conversational process.

| Future purpose: use for chat translation.

Requirements overview

Streamline human tasks:

- Text analysis and categorization of repair descriptions
- Service category suggestion based on garment type and damage description
- Integration with React frontend for real-time suggestions
- Structured data output for consistent order processing
- Should be able to visualize data (components for visualizing data)

Usecase(s)

1. Employee wants to create an order in FikseSHOP. Goes to "Create order", and chooses to do so with AI assistance. The employee provides a prompt describing what has to be fixed (garment, fault/ flaw) and the AI agent suggests items and services which should make up the order.

User experience:

Employee: "Help me create an order for this damaged silk dress"

Agent: "I can help! Can you describe the damage you see?"

Employee: "There's a small tear near the hem"

Agent: "Based on that, I suggest these repair services: [list]"

--List of service categories

--Employee select the services

Agent: "Do you want to add other services manually?"

Employee: No

Agent: Do you confirm the order?

Employee : Yes

--Order is created.

Agent: "Your order is created, Here is your order summary"

- List of service categories (5 list) should pop up and then employee can select the services.
 - Ai should ask whether you want It should be able to create an order for the customer
1. Future Usecase : Employee wants contact details of any particular person in database. A separate dataset with the employee registry should be created with the phone number, address etc. And then AI should search the database and produce 1 good result.
 2. Future Usecase : Marketing campaign . Employee want to create a summer marketing campaign for dry cleaning on dresses and suits."

For this project below is my python code:

"# This is the hybrid agent, it uses LLM(ollama model) for intent detection and response generation.

It also uses the search module.

agent_hybrid.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import re
import requests
from typing import List, Dict, Optional
import uuid
from datetime import datetime
import json

app = FastAPI()
```

1. Data Models

```
-----  
  
class AgentInput(BaseModel):  
    session_id: str  
    user_input: str  
  
class ServiceItem(BaseModel):  
    id: str  
    service: str  
    description: str  
    price: float  
    garment_type: str  
    repairer_type: str  
    estimated_hours: Optional[float] = None  
  
class OrderSummary(BaseModel):  
    order_id: str  
    services: List[ServiceItem]  
    total_price: float  
    estimated_total_hours: Optional[float] = None  
    created_at: str  
  
-----
```

2. Enhanced Intent Detection

```
-----  
  
def detect_intent_and_context(text: str) → Dict:  
    """Enhanced intent detection with AI fallback for unknown intents"""  
    text_lower = text.lower()  
    print(f"🔍 Intent detection for: '{text}'")
```

```
# Extract context first (always useful)  
context = {  
    "garment_type": None,  
    "damage_type": None,
```

```

    "fabric_type": None
}

# Detect garment types (expanded list)
garments = ["dress", "shirt", "pants", "jacket", "coat", "blouse", "skirt", "suit",
            "jeans", "trousers", "sweater", "cardigan", "blazer", "shorts", "top",
            "outfit", "clothing", "garment", "clothes"]
for garment in garments:
    if garment in text_lower:
        context["garment_type"] = garment
        print(f"👗 Found garment: {garment}")
        break

# Detect fabric types
fabrics = ["silk", "cotton", "wool", "linen", "polyester", "denim", "leather",
           "cashmere", "satin", "chiffon", "velvet", "corduroy"]
for fabric in fabrics:
    if fabric in text_lower:
        context["fabric_type"] = fabric
        print(f"🧵 Found fabric: {fabric}")
        if context["garment_type"]:
            context["garment_type"] = f"{fabric} {context['garment_type']}"
            break

# Detect damage/issue types (expanded list)
damage_types = ["tear", "hole", "stain", "zipper", "button", "seam", "hem", "rip",
                "worn", "faded", "shrunk", "stretched", "loose", "tight", "broken",
                "damaged", "ruined", "falling apart", "needs fixing"]
for damage in damage_types:
    if damage in text_lower:
        context["damage_type"] = damage
        print(f"💔 Found damage: {damage}")
        break

# Simple intent patterns first

```

```

if re.match(r"^[0-9]+$", text.strip()):
    print("📄 Matched number pattern")
    return {"intent": "service_selection", "context": context}
elif re.search(r'\b(yes|confirm|okay|ok)\b|looks good', text_lower):
    print("✅ Matched confirmation pattern")
    return {"intent": "confirmation", "context": context}
elif re.search(r'\b(no|cancel|nevermind|back)\b', text_lower):
    print("❌ Matched cancel pattern")
    return {"intent": "cancel", "context": context}
elif re.search(r'\b(hi|hello|hey|start|begin)\b', text_lower):
    match = re.search(r'\b(hi|hello|hey|start|begin)\b', text_lower)
    matching_phrase = match.group(1) if match else "unknown"
    print(f"👋 Matched greeting pattern: '{matching_phrase}' as whole word")
    return {"intent": "greeting", "context": context}

# If we found garment/fabric/damage context, likely a repair request
print(f"📄 Context extracted: {context}")
if context["garment_type"] or context["fabric_type"] or context["damage_type"]:
    print("✅ Detected repair_request from context")
    return {"intent": "repair_request", "context": context}

# For everything else, use AI to classify intent
print("🤖 Using AI to classify intent")
return ai_classify_intent(text, context)

```

```

def ai_classify_intent(text: str, context: Dict) → Dict:
    """Use AI to classify intent when keyword matching fails"""
    try:
        prompt = f"""You are an intent classifier for a clothing repair service.
        User said: "{text}"

        Based on this input, classify the intent as one of:
        • repair_request: User needs clothing repair/alteration/fixing
        • greeting: User is saying hello or starting conversation

```

- service_selection: User is selecting from options
- confirmation: User is confirming something
- unknown: Doesn't fit any category

Respond with ONLY the intent name, nothing else."""

```
response = requests.post("<http://localhost:11434/api/generate>", json={
    "model": "phi3",
    "prompt": prompt,
    "stream": False
})

ai_intent = response.json()["response"].strip().lower()
print(f"🤖 AI classified intent as: '{ai_intent}'")

# Validate AI response
valid_intents = ["repair_request", "greeting", "service_selection", "confirmati
on", "unknown"]
if ai_intent in valid_intents:
    print(f"✅ Valid AI intent: {ai_intent}")
    return {"intent": ai_intent, "context": context}
else:
    print(f"❌ Invalid AI intent: {ai_intent}, using fallback")
    # Fallback: if AI response is invalid, assume repair request if any context
found
    if context["garment_type"] or context["fabric_type"] or context["damage_
type"]]:
        return {"intent": "repair_request", "context": context}
    return {"intent": "unknown", "context": context}

except Exception:
    # If AI fails, use smart fallback
    if context["garment_type"] or context["fabric_type"] or context["damage_ty
pe"]]:
```

```
    return {"intent": "repair_request", "context": context}
    return {"intent": "unknown", "context": context}
```

3. AI Response Generation

```
class AIResponseGenerator:
    def
    init(self, model_name="phi3"):
        self.model_name = model_name

    def generate_response(self, intent: str, context: Dict, user_input: str, session_data: Dict = None) → str:
        """Generate AI response based on intent and context"""

        # Create context-aware prompts
        if intent == "repair_request":
            prompt = self._create_repair_prompt(context, user_input, session_data)
        elif intent == "greeting":
            prompt = self._create_greeting_prompt(context, user_input, session_data)
        else:
            prompt = self._create_general_prompt(context, user_input, session_data)

        return self._call_ollama(prompt)

    def _create_repair_prompt(self, context: Dict, user_input: str, session_data: Dict) → str:
        garment_info = f"{context['fabric_type']} {context['garment_type']}" if context.get('fabric_type') and context.get('garment_type') else context.get('garment_type', 'garment')
        services_count = session_data.get('services_found', 0) if session_data else 0
```

```
if services_count > 0:
    return f""Generate a short, direct response (maximum 15 words) that says you found matching repair services for their {garment_info}.
```

Example: "Found {services_count} matching repair services for your {garment_info}. Here are your options:"

Keep it brief and direct. No explanations or extra details."""

else:

```
return f""Generate a short response asking for more details about the {garment_info} damage. Maximum 20 words.
```

Example: "Could you describe the damage to your {garment_info} in more detail?"

Keep it brief."""

```
def _create_greeting_prompt(self, context: Dict, user_input: str, session_data: Dict) → str:
    return f""Generate a brief greeting (maximum 25 words) for a clothing repair service.
```

Example: "Hi! I help with clothing repairs and alterations. What garment needs fixing today?"

Keep it short and friendly."""

```
def _create_general_prompt(self, context: Dict, user_input: str, session_data: Dict) → str:
    context_clues = []
    if context["garment_type"]:
        context_clues.append(f"garment: {context['garment_type']}")
    if context["fabric_type"]:
        context_clues.append(f"fabric: {context['fabric_type']}")
    if context["damage_type"]:
        context_clues.append(f"issue: {context['damage_type']}")

    context_info = f" I noticed you mentioned: {' '.join(context_clues)}." if context_clues else ""
```

```
return f"""You are a helpful AI assistant for a clothing repair service.{context_info}
```

User said: "{user_input}"

Keep it short and direct."""

```
def _call_ollama(self, prompt: str) → str:
    """Call Ollama API for response generation"""
    try:
        response = requests.post("<http://localhost:11434/api/generate>", json={
            "model": self.model_name,
            "prompt": prompt,
            "stream": False
        })
        response.raise_for_status()
        return response.json()["response"].strip()
    except Exception as e:
        return f"I apologize, but I'm having trouble generating a response right now. Please describe what clothing item needs repair and I'll do my best to help!"
```

4. Session Management

```
sessions = {}

class SessionState:
    def
    init(self):
        self.user_name: Optional[str] = None
        self.conversation_state: str = "greeting" # greeting, searching, selecting,
        confirming, completed
```

```

self.conversation_history: List[Dict] = []
self.context: Dict = {}
self.suggested_services: List[ServiceItem] = []
self.selected_services: List[ServiceItem] = []
self.pending_order: Optional[OrderSummary] = None
self.current_query: Optional[str] = None

def get_session(session_id: str) → SessionState:
if session_id not in sessions:
sessions[session_id] = SessionState()
return sessions[session_id]

```

5. Service Functions

```

def query_fikse_search(query: str) → List[ServiceItem]:
"""Query the search service for repair services"""
try:
print(f"🔍 Searching for: {query}")
response = requests.get("
http://localhost:8000/search", params={"q": query})
response.raise_for_status()
results = response.json()
print(f"📄 Found {len(results)} raw results")

```

```

services = []
for i, result in enumerate(results[:10]):
    service_item = ServiceItem(
        id=f"service_{i+1}",
        service=result.get("Service", "Unknown Service"),
        description=result.get("Description", ""),
        price=float(result.get("Price", 0)),
        garment_type=result.get("Type of garment in category", ""),
        repairer_type=result.get("Type of Repairer", ""),

```

```

        estimated_hours=result.get("Estimated time in hours")
    )
    services.append(service_item)

    print(f"✅ Returning {len(services)} formatted services")
    return services
except Exception as e:
    print(f"❌ Search error: {str(e)}")
    return []

```

6. Main Agent Logic

```

ai_generator = AIResponseGenerator()

@app.post("/agent")
def hybrid_agent(input: AgentInput):
    """Hybrid agent that combines intent detection with AI generation"""
    try:
        session = get_session(input.session_id)
        print(f"🇮🇹 Session state: conversation_state={session.conversation_state},
        suggested_services={len(session.suggested_services)}, selected_services=
        {len(session.selected_services)}")

```

```

        # Detect intent and context
        intent_data = detect_intent_and_context(input.user_input)
        intent = intent_data["intent"]
        context = intent_data["context"]
        print(f"🎯 Final intent: {intent}, context: {context}")

        # Update session context
        session.context.update(context)

```

```

        # Add to conversation history

```

```

session.conversation_history.append({
    "role": "user",
    "content": input.user_input,
    "intent": intent,
    "context": context
})

# Handle repair requests
if intent == "repair_request":
    # Search for services
    services = query_fikse_search(input.user_input)
    session.suggested_services = services[:5]
    session.conversation_state = "selecting" if services else "searching"
    session.current_query = input.user_input

# Generate direct response
if services:
    # Fix garment_info formatting to avoid duplicates
    fabric = context.get('fabric_type', '')
    garment = context.get('garment_type', 'garment')
    if fabric and garment and fabric in garment:
        garment_info = garment # Already contains fabric
    elif fabric and garment:
        garment_info = f"{fabric} {garment}"
    else:
        garment_info = garment or fabric or "garment"

    response_text = f"Found {len(session.suggested_services)} matching r
    epair services for your {garment_info}. Here are your options:"
    else:
        garment_info = context.get('garment_type', 'item')
        response_text = f"I couldn't find services for your {garment_info}. Coul
        d you describe the damage in more detail?"

    return {
        "intent": intent,

```

```

        "response": response_text,
        "conversation_state": session.conversation_state,
        "show_services": len(services) > 0,
        "services": [s.dict() for s in session.suggested_services],
        "context": context
    }

# Handle service selection
elif intent == "service_selection":
    print(f"📝 Service selection: user input = '{input.user_input}'")
    print(f"📋 Available services: {len(session.suggested_services)}")

    if session.suggested_services:
        try:
            # Parse service selection (expecting numbers like "1", "2", etc.)
            selection_number = int(input.user_input.strip())
            print(f"🎯 Parsed selection: {selection_number}")

            if 1 <= selection_number <= len(session.suggested_services):
                selected_service = session.suggested_services[selection_number
- 1]

                session.selected_services = [selected_service]
                session.conversation_state = "confirming"

                # Create order summary preview for confirmation
                order_preview = OrderSummary(
                    order_id="PREVIEW", # Temporary ID
                    services=[selected_service],
                    total_price=selected_service.price,
                    estimated_total_hours=selected_service.estimated_hours,
                    created_at="" # Will be set when actually confirmed
                )
                session.pending_order = order_preview

                print(f"✅ Selected service: {selected_service.service} - ${selected_service.price}")

```

```

print(f"🔄 Updated conversation_state to: {session.conversation_s
tate}")

print(f"📋 Created order preview for UI")

response_text = f"Great choice! You've selected:\n\n**{selected
_service.service}** - ${selected_service.price:.0f}\n\n{selected_service.descri
ption}\n\n**Click the confirmation buttons below or type 'yes' to confirm.**"

return {
    "intent": intent,
    "response": response_text,
    "conversation_state": "confirming",
    "show_services": False,
    "selected_services": [selected_service.dict()],
    "order_summary": order_preview.dict(), # This is what the UI ne
eds!

    "context": context
}
else:
    print(f"❌ Invalid selection: {selection_number} not in range 1-{len
(session.suggested_services)}")
    except (ValueError, IndexError) as e:
        print(f"❌ Parse error: {e}")
    else:
        print(f"❌ No suggested services in session")

# Fallback if selection fails
print(f"⚠️ Service selection failed - using fallback response")
response_text = ai_generator.generate_response("unknown", context, inp
ut.user_input)
return {
    "intent": "unknown",
    "response": response_text,
    "conversation_state": session.conversation_state,
    "show_services": len(session.suggested_services) > 0,
    "services": [s.dict() for s in session.suggested_services],

```

```

        "context": context
    }

    # Handle confirmation
    elif intent == "confirmation" and session.conversation_state == "confirming":
        if session.selected_services:
            # Create final order with real ID and timestamp
            final_order = OrderSummary(
                order_id=f"ORD-{uuid.uuid4().hex[:8].upper()}",
                services=session.selected_services,
                total_price=sum(s.price for s in session.selected_services),
                estimated_total_hours=sum(s.estimated_hours for s in session.selected_services if s.estimated_hours),
                created_at=datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            )
            session.pending_order = final_order
            session.conversation_state = "completed"

            print(f"🎉 Order confirmed! ID: {final_order.order_id}")

            response_text = f"🎉 **Order Created Successfully!**\n\n**Order ID: {final_order.order_id}\n**Service:** {session.selected_services[0].service}\n\n**Price:** ${final_order.total_price:.0f}\n\n**Created:** {final_order.created_at}\n\nYour repair order is ready for processing! Is there anything else I can help you with?"

        return {
            "intent": intent,
            "response": response_text,
            "conversation_state": "completed",
            "show_services": False,
            "order_created": final_order.dict(),
            "context": context
        }

```

```

# Handle cancellation
elif intent == "cancel":
    if session.conversation_state == "confirming":
        # Reset to service selection
        session.conversation_state = "selecting"
        session.selected_services = []
        session.pending_order = None # Clear the order preview
        print("❌ Order cancelled, reset to selecting state")
        response_text = "Order cancelled. Please select a different service by t
yping its number (1, 2, 3, etc.):"

    return {
        "intent": intent,
        "response": response_text,
        "conversation_state": "selecting",
        "show_services": len(session.suggested_services) > 0,
        "services": [s.dict() for s in session.suggested_services],
        "context": context
    }
else:
    # General reset
    session.conversation_state = "greeting"
    response_text = "No problem! What clothing item would you like to get
repaired?"

    return {
        "intent": intent,
        "response": response_text,
        "conversation_state": "greeting",
        "show_services": False,
        "context": context
    }

# Handle greetings with direct response
elif intent == "greeting":
    session.conversation_state = "greeting"

```

```
response_text = "Hi! How can I help you today?"
```

```
return {  
    "intent": intent,  
    "response": response_text,  
    "conversation_state": "greeting",  
    "show_services": False,  
    "context": context  
}
```

```
# Generate general AI response for other intents
```

```
response_text = ai_generator.generate_response(intent, context, input.user  
_input)
```

```
return {  
    "intent": intent,  
    "response": response_text,  
    "conversation_state": session.conversation_state,  
    "show_services": False,  
    "context": context  
}
```

```
except Exception as e:
```

```
    return {  
        "intent": "error",  
        "response": "I apologize, but I'm having trouble right now. Please describ  
e what clothing item needs repair and I'll try to help!",  
        "conversation_state": "greeting",  
        "show_services": False,  
        "error": str(e)  
    }
```

```
@app.get("/")
```

```
def root():
```

```
    return {"message": "AI-Powered Clothing Repair Agent", "specialty": "clothing  
repair and alteration services"}
```

```
@app.get("/health")
def health_check():
    return {"status": "healthy", "service": "clothing-repair-agent", "ai_model": "phi3"}
"
```

This was 1 month project, divide this project into tasks and then subtasks